

Resumen Teórico: Complejidad Computacional

Computabilidad y Complejidad — UNRC

Basado en el material de Pablo Castro (UNRC)

Índice

1. Introducción	3
2. Tiempo de ejecución y notaciones asintóticas	3
2.1. Tiempo como función	3
2.2. Notación O (big-oh)	3
2.3. Notación o (little-oh)	3
2.4. Notación Ω (omega)	4
2.5. Notación Θ (theta)	4
2.6. Relación entre notaciones	4
3. Análisis de tiempo: un ejemplo guía	4
3.1. Primer algoritmo: $O(n^2)$	4
3.2. Segundo algoritmo: $O(n \log n)$	5
3.3. Tercer algoritmo: dos cintas, $O(n)$	5
4. Clases de complejidad temporal y la clase P	5
4.1. Preguntas centrales de complejidad	5
4.2. Crecimiento polinomial vs exponencial	6
4.3. Definición de la clase P	6
5. Modelos de computación y la Tesis de Church–Turing Extendida	6
5.1. Modelos considerados	6
5.2. MT multicinta vs MT de una cinta	6
5.3. Tesis de Church–Turing Extendida	7
6. Máquinas no deterministas y su simulación	7
6.1. Tiempo en una MT no determinista	7
6.2. Simulación de una MT no determinista	7
7. La clase P: ejemplos	7
7.1. PATH	7
7.2. Coprimos	8
7.3. Gramáticas Libres de Contexto	8
8. La clase NP	8
9. Caminos Hamiltonianos como ejemplo guía	9
9.1. Solución no determinista para HAMPATH	9
10. Verificadores y certificados	9

10.1. Verificabilidad polinomial	9
10.2. Definición formal	10
10.3. Definición alternativa de NP	10
11. Ejemplos de problemas en NP	10
11.1. HAMPATH	10
11.2. COMP (números compuestos)	10
11.3. CLIQUE	11
11.4. SUBSET-SUM	11
11.5. Problemas afuera de NP	11
12. P vs NP	11
13. Reducibilidad polinomial	12
13.1. Propiedades de la reducibilidad polinomial	12
14. NP-Compleitud	12
15. SAT, 3-SAT y el Teorema de Cook–Levin	13
15.1. Fórmulas booleanas	13
15.2. CNF y 3-CNF	13
15.3. Teorema de Cook–Levin	13
15.4. La tabla de cómputo	14
15.5. Variables proposicionales	14
15.6. Estructura de ϕ	14
15.7. Tamaño total y correctitud	15
15.8. 3-SAT es NP-completo	15
16. Reducciones desde 3-SAT	16
16.1. $3\text{-SAT} \leq_P \text{CLIQUE}$	16
16.2. $3\text{-SAT} \leq_P \text{VERTEX-COVER}$	16
16.3. $3\text{-SAT} \leq_P \text{HAMPATH}$	17
16.4. Otros NP-completos clásicos	17
17. Propiedades de clausura	17
18. Complejidad espacial	17
18.1. Definición formal	17
18.2. Clases de complejidad espacial	17
18.3. $\text{SAT} \in \text{PSPACE}$	18
18.4. $\overline{\text{ALL}_{\text{NFA}}} \in \text{NPSpace}$	18
19. El Teorema de Savitch	18
19.1. El algoritmo <code>canyield</code>	18
19.2. Análisis	19
19.3. Consecuencias	19
20. Jerarquía $P \subseteq NP \subseteq \text{PSPACE} \subseteq \text{EXPTIME}$ y PSPACE-Compleitud	19
20.1. Cadena de inclusiones	19
20.2. PSPACE-Compleitud	20
21. Ideas clave para repasar	20
22. Preguntas típicas de examen	21

1. Introducción

En *computabilidad* la pregunta central es: *dado un problema P , ¿es computable?*. En *complejidad*, en cambio, ya partimos de problemas computables y la pregunta cambia a: *¿hay algoritmos eficientes para resolver P ?*

Surge entonces un interrogante metodológico: *¿cuál modelo de computación usamos para evaluar si un algoritmo es eficiente o no?* Esta cuestión es crucial; la respuesta — vía la **Tesis de Church–Turing Extendida** — justificará usar máquinas de Turing como modelo de referencia.

Intuición

Computabilidad pregunta “¿hay algoritmo?”; complejidad pregunta “¿hay algoritmo cuyo tiempo crezca razonablemente con el tamaño de la entrada?”. Ambas son preguntas matemáticas, pero la segunda introduce un parámetro adicional: el tamaño n de la entrada y una función $T(n)$ que mide cuánto trabajo se realiza.

2. Tiempo de ejecución y notaciones asintóticas

2.1. Tiempo como función

Dado un algoritmo (una MT) M , su tiempo de ejecución se modela como una función

$$T_M : \mathbb{N} \rightarrow \mathbb{R}^{\geq 0}$$

que asigna, a cada n , la cantidad de pasos que M realiza sobre alguna entrada de tamaño n . Convenciones estándar:

- T se asume **monótona**.
- M es una MT que **termina en todas sus entradas**.
- En general el análisis se hace en el **peor caso**.
- Interesa cómo crece T cuando n se hace grande (comportamiento asintótico).

2.2. Notación O (big-oh)

Definición 2.1 (Notación O).

$$f(n) \in O(g(n)) \iff \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) \leq c \cdot g(n).$$

Intuición

Para entradas grandes, g crece igual o más rápido que f módulo constantes multiplicativas. Es una **cota superior** asintótica.

2.3. Notación o (little-oh)

Definición 2.2 (Notación o).

$$f(n) \in o(g(n)) \iff \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) < c \cdot g(n).$$

Equivalentemente:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \iff f(n) \in o(g(n)).$$

Observación 2.3. g crece *estrictamente* más rápido que f . En particular *nunca* ocurre $f(n) \in o(f(n))$.

2.4. Notación Ω (omega)

Definición 2.4 (Notación Ω).

$$f(n) \in \Omega(g(n)) \iff \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) \geq c \cdot g(n).$$

Intuición

Para entradas grandes, f crece igual o más rápido que g módulo constantes. Es una **cota inferior** asintótica.

2.5. Notación Θ (theta)

Definición 2.5 (Notación Θ).

$$f(n) \in \Theta(g(n)) \iff f(n) \in O(g(n)) \text{ y } f(n) \in \Omega(g(n)).$$

Intuición

f y g crecen *igual* módulo constantes: es una cota **ajustada**.

2.6. Relación entre notaciones

Notación	Significado	Tipo de cota
$f \in O(g)$	f crece a lo sumo como g	Superior
$f \in o(g)$	f crece estrictamente menos que g	Superior estricta
$f \in \Omega(g)$	f crece al menos como g	Inferior
$f \in \Theta(g)$	f y g crecen igual	Ajustada

3. Análisis de tiempo: un ejemplo guía

Sea el lenguaje

$$A = \{0^k 1^k \mid k \geq 0\}.$$

Veremos tres algoritmos para decidirlo con distinta complejidad.

3.1. Primer algoritmo: $O(n^2)$

Dado el input w :

1. Se recorre la cinta; si se encuentra “10” se rechaza.
2. Se vuelve al principio, se tacha el primer 0 y se busca el primer 1 y se tacha.
3. Se repite el paso 2 hasta que:
 - No queden 0s ni 1s — *acepta*.
 - Queden solo 0s o solo 1s — *rechaza*.

Análisis. El paso 1 hace $O(n)$ pasos. El paso 2, cada pasada, $O(n)$. El paso 3 se repite $\sim n/2$ veces. Total: $O(n) + O(n/2 \cdot n) \in O(n^2)$.

3.2. Segundo algoritmo: $O(n \log n)$

Idea: usar paridad.

1. Recorrer la cinta; si se encuentra “10”, rechazar.
2. Repetir mientras haya 0s o 1s:
 - a) Contar la cantidad de 0s y 1s; si alguna es impar, rechazar.
 - b) Cada dos 0s, tachar uno (alternando), y lo mismo para los 1s.
3. Si no quedan 0s ni 1s, aceptar; sino rechazar.

Ejemplo 3.1. Con $w = 0^{25}1^{25}$: la cantidad es par; tras tachar la mitad quedan 12 ceros y 12 unos; luego 6 y 6; luego 3 y 3; luego 2 y 2; luego 1 y 1. En cada iteración el número se *divide por 2*, así que hay $O(\log n)$ iteraciones, cada una de costo $O(n)$. Total: $O(n \log n)$.

3.3. Tercer algoritmo: dos cintas, $O(n)$

Con una MT de dos cintas:

1. Recorrer la cinta; si encuentra “10”, rechazar.
2. Recorrer la cinta y copiar los ceros a la segunda cinta.
3. Recorrer la cinta otra vez; cada vez que aparezca un 1, tachar un cero en la segunda cinta. Si no hay más ceros, rechazar.
4. Si todos los ceros de la segunda cinta fueron tachados, aceptar; sino rechazar.

Total: 4 fases lineales $\Rightarrow O(n)$.

Idea clave

Un mismo problema admite algoritmos de tiempo muy diferente según el modelo y la astucia: $O(n^2)$, $O(n \log n)$, $O(n)$. Esto motiva preguntas finas: ¿cuál es el *mínimo* tiempo posible? ¿importa el modelo de cómputo?

4. Clases de complejidad temporal y la clase P

Definición 4.1 (Clase TIME).

$$\text{TIME}(t(n)) = \{L \mid L \text{ se puede decidir con una MT en } O(t(n)) \text{ pasos}\}.$$

Por ejemplo, $\text{TIME}(O(n^2))$ contiene los problemas para los cuales existe un algoritmo que, en el peor caso, realiza $c \cdot n^2$ pasos.

4.1. Preguntas centrales de complejidad

- Dado un problema, ¿pertenece a $\text{TIME}(O(t(n)))$?
- ¿Hay problemas en $\text{TIME}(O(t(n)))$ pero *no* en $\text{TIME}(O(t'(n)))$? (Jerarquías.)

4.2. Crecimiento polinomial vs exponencial

	$n = 10$	$n = 20$	$n = 30$	$n = 40$	$n = 50$
n	0.00001 s	0.00002 s	0.00003 s	0.00004 s	0.00005 s
n^2	0.0002 s	0.0004 s	0.0009 s	0.0016 s	0.0025 s
n^3	0.1 s	3.2 s	24.3 s	1.7 min	15.2 min
2^n	0.001 s	1.0 s	17.9 min	12.7 días	35.6 años

Idea clave

La diferencia entre polinomial y exponencial no es estética: para $n = 50$ el algoritmo n^3 tarda 15 minutos y el 2^n tarda 35 años. Por eso se consideran **tratables** los problemas con algoritmo en $O(n^k)$.

4.3. Definición de la clase P

Definición 4.2 (Clase P).

$$P = \bigcup_{k \geq 0} \text{TIME}(O(n^k)).$$

Propiedades clave:

- P es *invariante* respecto a los modelos deterministas razonables.
- Si un problema tiene solución polinomial, los avances en hardware tienen impacto significativo.

5. Modelos de computación y la Tesis de Church–Turing Extendida

5.1. Modelos considerados

- MTs deterministas con una sola cinta.
- MTs deterministas con varias cintas.
- MTs deterministas con cinta infinita en ambos lados.
- MTs no deterministas.

Las tres primeras se simulan entre sí en tiempo polinomial. La clase TIME toma como referencia las máquinas deterministas.

5.2. MT multicinta vs MT de una cinta

Teorema 5.1. Cualquier MT con muchas cintas que hace $O(t(n))$ pasos puede ser traducida a una MT de una cinta que la simula en $O(t(n)^2)$ pasos.

Idea. Se concatenan las k cintas en una sola, separadas por un símbolo #, y se utilizan símbolos especiales para marcar dónde están las cabezas. Por cada paso de la MT multicinta se recorre la cinta simulada (a lo sumo $k \cdot t(n)$ símbolos), realizando $O(t(n))$ pasos. Como hay $t(n)$ pasos a simular, el total es $O(t(n)^2)$. $\square$

5.3. Tesis de Church–Turing Extendida

Recordemos la tesis clásica:

Si un problema es computable por algún modelo de computación, entonces es computable en todos los otros.

La versión extendida agrega la dimensión temporal:

*Si un problema es decidable en tiempo $O(t(n))$ por algún modelo razonable de computación (**determinista**), entonces es decidable en $O(t(n)^k)$ en cualquier otro modelo razonable de computación (**determinista**).*

Idea clave

La tesis extendida justifica usar las MTs deterministas como modelo de referencia: si un problema tiene un algoritmo polinomial en cualquier modelo razonable, lo tiene en MT. Por eso P es robusta.

6. Máquinas no deterministas y su simulación

6.1. Tiempo en una MT no determinista

En una MT *determinista* la única ejecución debe terminar en a lo sumo $t(n)$ pasos. En una MT *no determinista* hay un árbol de cómputo y todas las ramas deben terminar en a lo sumo $t(n)$ pasos: el tiempo es la longitud de la *rama más larga*.

6.2. Simulación de una MT no determinista

Teorema 6.1. Cualquier MT no determinista que hace $O(t(n))$ pasos puede ser simulada por una MT determinista que hace $2^{O(t(n))}$ pasos.

Idea. Como la MT termina, el árbol tiene a lo sumo $|b|^{t(n)}$ nodos (con b el branching factor). En el peor caso hay que recorrerlos todos (con BFS por niveles para evitar perderse en una rama infinita). Como $|b|^{t(n)} \leq (2^c)^{t(n)} = 2^{c \cdot t(n)} \in 2^{O(t(n))}$. $\square$

Idea clave

El no determinismo permite “adivinar” decisiones, pero simularlo deterministamente cuesta una *explosión exponencial*. La pregunta $P \stackrel{?}{=} NP$ es exactamente la pregunta: ¿es realmente necesaria esa explosión?

7. La clase P: ejemplos

7.1. PATH

$\text{Path} = \{\langle G, v, w \rangle \mid G \text{ es un grafo con un camino de } v \text{ a } w\}$.

Algoritmo ineficiente. Para $i = 1, 2, \dots, |V|$, generar todos los posibles caminos de longitud i que empiezan en v y terminan en w , aceptando si alguno está en el grafo. Es $O(n!)$ con $n = |V|$.

Algoritmo eficiente (búsqueda en anchura).

1. Marcar v .

2. Mientras haya nodos sin marcar: recorrer los arcos de G ; para cada (a, b) con a marcado y b no, marcar b . Si se marca w , aceptar.
3. Si no se llegó, rechazar.

Costo: $O(|V| \cdot |E|)$, polinomial.

7.2. Coprimos

$$\text{Coprimos} = \{\langle x, y \rangle \mid x \text{ y } y \text{ son coprimos}\}.$$

Algoritmo ineficiente. Para todo z con $1 < z \leq \min(x, y)$, chequear si $z \mid x$ y $z \mid y$. Como los números vienen en binario, si tienen n bits hay que hacer 2^n pasos: *exponencial*.

Algoritmo eficiente (Euclides). Dados x, y en binario:

- Repetir hasta que $y = 0$: $x \leftarrow x \bmod y$; intercambiar x e y .
- Si $x = 1$, aceptar; sino rechazar.

En cada paso x se divide al menos por 2; con n bits se hacen n iteraciones. Cada división se computa en tiempo polinomial $p(n)$. Total: $n \cdot p(n)$, polinomial.

Observación 7.1. Cuidado con la codificación. El tamaño de la entrada para un número x es $\log_2 x$. Un algoritmo que itera “hasta x ” no es polinomial en el tamaño de la entrada.

7.3. Gramáticas Libres de Contexto

$$\text{GLC} = \{\langle G, w \rangle \mid w \text{ es generado por la gramática libre de contexto } G\}.$$

Usando programación dinámica y la *forma normal de Chomsky* (producciones $A \rightarrow BC$, $A \rightarrow a$, $A \rightarrow \varepsilon$) se construye una matriz $M[i][j]$ con los no-terminales que generan $w_i \cdots w_j$. Es el algoritmo **CYK**.

```
D = "Sobre input w = w_1 ... w_n:
  1. Si w = e, aceptar sii S -> e es regla.
  2. Para i = 1 hasta n:                                [substrings de longitud 1]
  3.   Para cada variable A:
  4.     Si A -> b es regla con b = w_i, poner A en table(i, i).
  5. Para l = 2 hasta n:                                [l = longitud del substring]
  6.   Para i = 1 hasta n - l + 1:
  7.     j = i + l - 1.
  8.     Para k = i hasta j - 1:                          [k = posicion de corte]
  9.       Para cada regla A -> BC:
  10.        Si table(i,k) contiene B y table(k+1,j) contiene C,
            poner A en table(i,j).
  11. Si S esta en table(1,n), aceptar; sino rechazar."
```

Complejidad: $O(|v| \cdot n^3)$ aproximadamente. Es polinomial $\Rightarrow \text{GLC} \in \text{P}$.

8. La clase NP

Al igual que con P, usamos MTs no deterministas para definir una clase:

Definición 8.1 (Tiempo no determinista).

$$\text{NTIME}(t(n)) = \{L \mid L \text{ es decidido por una MT no determinista en } O(t(n)) \text{ pasos}\}.$$

Definición 8.2 (Clase NP).

$$\text{NP} = \bigcup_{k>0} \text{NTIME}(n^k).$$

Intuición

Una MT no determinista en tiempo polinomial es “mágica”: en cada paso elige entre varias transiciones y acepta si *alguna* rama lleva a un estado de aceptación. Equivale a tener un oráculo que adivina la respuesta y luego solo hay que verificarla.

9. Caminos Hamiltonianos como ejemplo guía

Definición 9.1 (Camino Hamiltoniano). Dado un grafo dirigido $G = (V, E)$, un camino Hamiltoniano es un camino que pasa *exactamente una vez* por cada nodo.

Definición 9.2 (Lenguaje HAMPATH).

$$\text{HAMPATH} = \{\langle G, s, t \rangle \mid G \text{ tiene un camino Hamiltoniano de } s \text{ a } t\}.$$

9.1. Solución no determinista para HAMPATH

Dado $\langle G, s, t \rangle$ en la cinta de entrada:

1. **Adivinar.** Escribir, de forma no determinista, $|V|$ nodos.
2. **Verificar.** Comprobar que esos nodos están consecutivamente conectados, son todos distintos, $v_1 = s$ y $v_{|V|} = t$. Aceptar si es así, rechazar en caso contrario.

Idea clave

Patrón general para problemas en NP:

- **Etap 1 (adivinanza):** se adivina no deterministamente una posible solución (candidato).
- **Etap 2 (verificación):** se verifica deterministamente, en tiempo polinomial, si efectivamente resuelve el problema.

10. Verificadores y certificados

10.1. Verificabilidad polinomial

Decimos que una cadena c es un **certificado** (o testigo) de un problema si, junto con la entrada, permite ver la solución al problema.

HAMPATH tiene dos características importantes:

1. **Certificados cortos.** Un camino Hamiltoniano se puede describir en menos espacio que el grafo.
2. **Verificación rápida.** Dado un camino, podemos verificar en tiempo lineal si es solución o no.

Los problemas en NP poseen certificados polinomiales verificables polinomialmente.

10.2. Definición formal

Definición 10.1 (Verificador). Sea A un lenguaje. Una MT determinista V es un **verificador** para A si

$$A = \{w \mid \exists c : V \text{ acepta } (w, c)\}.$$

La cadena c se llama **certificado**.

Definición 10.2 (Verificador polinomial). Un verificador V se dice **polinomial** si V corre en tiempo $O(n^k)$ y los certificados son polinomialmente acotados:

$$A = \{w \mid \exists c : |c| \leq |w|^k \text{ y } V \text{ acepta } (w, c)\}.$$

10.3. Definición alternativa de NP

Teorema 10.3 (Caracterización de NP). $L \in \text{NP}$ si y solo si L tiene un verificador polinomial.

Esquema. ($\Leftarrow$) **De verificador a MT no determinista.** Construimos una MT no determinista N que, sobre la entrada w , adivina un certificado c con $|c| \leq |w|^k$ y simula $V(w, c)$. Ambas fases son polinomiales.

($\Rightarrow$) **De MT no determinista a verificador.** Construimos un verificador V que recibe (w, c) donde c codifica una rama de cómputo de N . V usa c para guiar la simulación y acepta si la rama acepta. $\square$

Idea clave

Dos visiones equivalentes de NP:

- **Operacional:** problemas decidibles en tiempo polinomial por una MT *no determinista*.
- **Declarativa:** problemas cuyas instancias positivas admiten una *prueba corta* verificable polinomialmente.

La declarativa es la más útil en la práctica: probar $L \in \text{NP}$ casi siempre significa exhibir un certificado natural.

11. Ejemplos de problemas en NP

11.1. HAMPATH

Ya visto: certificado = el camino; verificación lineal. **HAMPATH** $\in \text{NP}$.

11.2. COMP (números compuestos)

Ejemplo 11.1.

$$\text{COMP} = \{x \mid \exists p, q > 1 : x = p \cdot q\}.$$

Certificado. Un par (p, q) con $p, q > 1$ y $p \cdot q = x$.

Verificador. Chequea $p, q > 1$ y computa $p \cdot q$ (multiplicación polinomial en cantidad de bits) comparando con x . Como $p, q < x$, los certificados son polinomiales. **COMP** $\in \text{NP}$.

11.3. CLIQUE

Definición 11.2 (Clique). Dado un grafo no dirigido G , un **clique** es un subgrafo en el cual cada par de nodos está conectado. Un k -**clique** es un clique con k nodos.

$$\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ tiene un } k\text{-clique}\}.$$

Certificado. Un subconjunto $c \subseteq V$.

Verificador. Dado $\langle G, c \rangle$:

- Controlar que los nodos de c estén en G .
- Controlar que todos los pares de nodos en c tengan arcos entre ellos.
- Aceptar si ambas condiciones se cumplen.

El subconjunto tiene longitud polinomial. **CLIQUE** $\in$ NP.

11.4. SUBSET-SUM

Ejemplo 11.3.

$$\text{SUBSET-SUM} = \{\langle S, t \rangle \mid \exists S' \subseteq S : \sum S' = t\}.$$

Por ejemplo, $\langle \{4, 11, 16, 21, 27\}, 25 \rangle \in \text{SUBSET-SUM}$ porque $4 + 21 = 25$.

Certificado. El subconjunto $S' \subseteq S$. **Verificador.** Suma los elementos de S' y compara con t . Polinomial. **SUBSET-SUM** $\in$ NP.

11.5. Problemas afuera de NP

Observación 11.4. No todo problema tiene un certificado obvio. Considere los complementos:

$$\overline{\text{HAMPATH}} = \{w \mid w \notin \text{HAMPATH}\}, \quad \overline{\text{CLIQUE}} = \{w \mid w \notin \text{CLIQUE}\}.$$

Para mostrar que un grafo *tiene* un camino Hamiltoniano basta un camino; para mostrar que *no tiene* ninguno, no se conoce un certificado polinomial natural. Análogamente para CLIQUE. Estos problemas pertenecen a coNP y no se sabe si $\text{NP} = \text{coNP}$.

12. P vs NP

Podemos resumir:

- P: lenguajes *decididos* rápidamente.
- NP: lenguajes cuya pertenencia se puede *verificar* rápidamente.

Proposición 12.1. $\text{P} \subseteq \text{NP}$.

Demostración. Si $L \in \text{P}$, hay una MT determinista M que decide L en tiempo polinomial. Definimos un verificador V que ignora el certificado y simula $M(w)$. Esto es polinomial y muestra $L \in \text{NP}$. $\square$

Pregunta abierta: $P \stackrel{?}{=} NP$

¿Es $P = NP$? Es decir, ¿todo problema cuya solución se puede *verificar* rápidamente se puede también *resolver* rápidamente? Es uno de los grandes problemas no resueltos de la computación. *A priori*, la verificabilidad polinomial parece más potente que la decidibilidad polinomial.

13. Reducibilidad polinomial

Para comparar la dificultad relativa de dos problemas se introduce una noción de reducción análoga a las de computabilidad, pero con la exigencia de eficiencia.

Definición 13.1 (Función polinomialmente computable). Una función $f : \Sigma^* \rightarrow \Sigma^*$ es **polinomialmente computable** si existe una MT que la computa en tiempo polinomial.

Definición 13.2 (Reducibilidad polinomial). Un lenguaje A es **polinomialmente reducible** a B , notado $A \leq_P B$, si existe una función polinomialmente computable $f : \Sigma^* \rightarrow \Sigma^*$ tal que

$$\forall w : w \in A \iff f(w) \in B.$$

Intuición

$A \leq_P B$ significa que “resolver A no es sustancialmente más difícil que resolver B módulo un trabajo polinomial”. Si sabemos resolver B , podemos resolver A traduciendo cada instancia de A a una de B y usando el algoritmo de B .

13.1. Propiedades de la reducibilidad polinomial

Proposición 13.3. Si $A \leq_P B$ y $B \in P$, entonces $A \in P$.

Demostración. Sea M una MT que computa f en tiempo $O(n^k)$ y M_B una MT que decide B en tiempo $O(n^{k'})$. Construyamos N que decide A :

1. Sobre la entrada w , ejecutar $M(w)$ para obtener $f(w)$.
2. Ejecutar $M_B(f(w))$; aceptar si M_B acepta, rechazar si rechaza.

Como M corre en $O(|w|^k)$ pasos, $|f(w)| \leq |w|^k$. Paso 2: $O((|w|^k)^{k'}) = O(|w|^{k \cdot k'})$. Total polinomial. $\square$

Proposición 13.4 (Transitividad). Si $A \leq_P B$ y $B \leq_P C$, entonces $A \leq_P C$.

Observación 13.5. Análogamente: si $A \leq_P B$ y $B \in NP$, entonces $A \in NP$. Las reducciones polinomiales preservan “hacia atrás” la pertenencia a P y a NP .

14. NP-Complejidad

Dentro de NP hay problemas que son “los más difíciles”: si alguien encontrara un algoritmo polinomial para alguno de ellos, automáticamente tendría algoritmos polinomiales para todos los problemas de NP .

Definición 14.1 (NP-complejidad). Un lenguaje L es **NP-completo** si:

1. $L \in NP$.

2. Para todo $A \in \text{NP}$, $A \leq_P L$ (se dice que L es **NP-difícil** o *NP-hard*).

Proposición 14.2. Si algún problema NP-completo está en P , entonces $P = \text{NP}$.

Demostración. Sea L NP-completo con $L \in P$. Para cualquier $A \in \text{NP}$, $A \leq_P L$. Por la Proposición 13.3, $A \in P$. Así $\text{NP} \subseteq P$, y como $P \subseteq \text{NP}$, concluimos $P = \text{NP}$. $\square$

Idea clave

Los NP-completos son “el centro nervioso” de NP. Probar $P = \text{NP}$ se reduce a encontrar *un* algoritmo polinomial para *cualquiera* de ellos. Recíprocamente, probar $P \neq \text{NP}$ se reduce a mostrar que *ninguno* de ellos admite uno.

15. SAT, 3-SAT y el Teorema de Cook–Levin

15.1. Fórmulas booleanas

Definición 15.1 (Fórmulas booleanas). Sea $X = \{x, y, z, \dots\}$ un conjunto numerable de variables. La gramática es

$$\phi, \psi ::= X \mid \neg\phi \mid \phi \wedge \psi \mid \phi \vee \psi.$$

Definición 15.2 (Asignación y satisfacibilidad). Una **asignación** es una función $v : X \rightarrow \{0, 1\}$. Una fórmula ϕ es **satisfacible** si existe v que la hace verdadera ($v \models \phi$).

Definición 15.3 (Lenguaje SAT).

$$\text{SAT} = \{\langle \phi \rangle \mid \phi \text{ es una fórmula booleana satisfacible}\}.$$

15.2. CNF y 3-CNF

Definición 15.4 (Cláusula, CNF, 3-CNF). Una **cláusula** es una disyunción de literales:

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_n).$$

Una fórmula está en **CNF** si es una conjunción de cláusulas. Está en **3-CNF** si cada cláusula tiene exactamente 3 literales. El lenguaje **3-SAT** es SAT restringido a 3-CNF.

Ejemplo 15.5. $(x_0 \vee x_1 \vee x_3) \wedge (\neg x_0 \vee \neg x_1 \vee x_2)$ está en 3-CNF.

Observación 15.6. Toda fórmula booleana se puede escribir en CNF.

15.3. Teorema de Cook–Levin

Teorema 15.7 (Cook–Levin). SAT es NP-completo.

La demostración tiene dos partes.

Parte 1: $\text{SAT} \in \text{NP}$. El certificado es una asignación v . El verificador evalúa ϕ bajo v en tiempo polinomial.

Parte 2: todo $A \in \text{NP}$ se reduce a SAT. Dado $A \in \text{NP}$ decidido por una MT no determinista N en tiempo n^k , hay que construir, en tiempo polinomial, una fórmula ϕ_w tal que

$$w \in A \iff \phi_w \text{ es satisfacible.}$$

15.4. La tabla de cómputo

Consideramos una matriz de $n^k \times n^k$ celdas:

- Cada **fila** representa una configuración (cinta + estado + cabezal).
- Hay n^k filas (una por paso de tiempo) y n^k columnas.
- Cada celda contiene un símbolo de $C = \Gamma \cup \Sigma \cup Q \cup \{\#\}$.

Se pasa de la fila i a la fila $i + 1$ si del estado en la fila i existe una ejecución de N que llega a la $i + 1$.

15.5. Variables proposicionales

Para cada celda (i, j) y símbolo $s \in C$ introducimos

$$x_{i,j,s} \text{ verdadera} \iff cell[i][j] = s.$$

Cantidad de variables: $|C| \cdot n^{2k} = O(n^{2k})$.

15.6. Estructura de ϕ

$$\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}.$$

ϕ_{cell} : cada celda tiene exactamente un símbolo

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s, t \in C \\ s \neq t}} (\neg x_{i,j,s} \vee \neg x_{i,j,t}) \right) \right].$$

Tamaño $O(n^{2k})$.

ϕ_{start} : la primera fila es la configuración inicial

$$\phi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge \cdots \wedge x_{1,n+2,w_n} \wedge x_{1,n+3,\sqcup} \wedge \cdots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#}.$$

Tamaño $O(n^k)$.

ϕ_{accept} : alguna fila contiene el estado de aceptación

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}.$$

Tamaño $O(n^{2k})$.

ϕ_{move} : cada paso es una transición válida

Idea: **ventanas de 2×3 celdas**. Una ventana consta de 6 celdas (3 en la fila i , 3 en la fila $i + 1$). Es *legal* si su contenido es consistente con alguna transición posible de N (incluyendo el caso de celdas que no cambian).

Teorema 15.8 (Ventanas y ramas de cómputo). Si todas las ventanas 2×3 son legales, entonces la matriz describe una rama de cómputo válida de N .

Intuición. Si un símbolo está en el centro de una ventana sin ser adyacente a un estado, debe mantenerse igual en la próxima fila. Si una ventana contiene un estado-cabezal en el centro, las filas inferiores se restringen a las que corresponden a alguna transición $\delta(q, a)$.

Ejemplo de ventanas legales. Si $\delta(q_1, a) = \{(q_1, b, R)\}$ y $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$, las siguientes son legales:

a	q_1	b
q_2	a	c

a	q_1	b
a	a	q_2

a	a	q_1
a	a	b

Entonces:

$$\phi_{\text{move}} = \bigwedge_{0 < i, j < n^k} (\text{la ventana centrada en } (i, j) \text{ es legal}).$$

Tamaño $O(n^{2k})$.

15.7. Tamaño total y correctitud

$$|\phi| = |\phi_{\text{cell}}| + |\phi_{\text{start}}| + |\phi_{\text{move}}| + |\phi_{\text{accept}}| = O(n^{2k}).$$

Polinomial. La construcción es polinomialmente computable.

Correctitud. Si N acepta w , hay una rama aceptante; codifica una tabla cuya asignación satisface ϕ . Si ϕ es satisfacible, la asignación describe una matriz que cumple las cuatro condiciones, por lo cual es una rama aceptante de N sobre w .

Idea clave

Una vez probado Cook–Levin, demostrar que un nuevo L es NP-completo no requiere repetir todo el argumento. Basta:

1. Mostrar $L \in \text{NP}$.
2. Reducir polinomialmente algún problema NP-completo conocido (SAT, 3-SAT, ...) a L .

La transitividad de $\leq_P$ cierra el círculo.

15.8. 3-SAT es NP-completo

Teorema 15.9. 3-SAT es NP-completo.

Esquema. (1) 3-SAT $\in \text{NP}$: idéntico a SAT.

(2) SAT $\leq_P$ 3-SAT. Como la fórmula de Cook–Levin se puede escribir en CNF, basta reducir CNF a 3-CNF. Dada

$$(x_0 \vee x_1 \vee \cdots \vee x_{n-1})$$

con n literales, introducimos $n - 2$ variables auxiliares $z_1, \dots, z_{n-2}$ y la reescribimos como:

$$(x_0 \vee x_1 \vee z_1) \wedge (\neg z_1 \vee x_2 \vee z_2) \wedge \cdots \wedge (\neg z_{n-2} \vee x_{n-2} \vee x_{n-1}).$$

Equisatisfacibilidad. Si la cláusula original se satisface con algún x_i verdadero, podemos asignar los z_j adecuadamente. Recíprocamente, si las nuevas cláusulas se satisfacen, por inducción alguna x_i debe ser verdadera.

Tamaño y tiempo. La fórmula tiene tamaño $O(N)$ y se computa en tiempo polinomial. Por transitividad, 3-SAT es NP-completo. $\square$

16. Reducciones desde 3-SAT

16.1. $3\text{-SAT} \leq_P \text{CLIQUE}$

Dada $\phi = (a_1 \vee b_1 \vee c_1) \wedge \cdots \wedge (a_k \vee b_k \vee c_k)$ en 3-CNF, construimos G_ϕ :

- Por cada cláusula generamos un *grupo* de 3 nodos (uno por literal).
- Conectamos cada par de nodos con un arco *excepto* si:
 - Pertenecen a la misma cláusula (mismo grupo).
 - Son contradictorios entre sí (x y $\neg x$).

Teorema 16.1. ϕ es satisfacible si y solo si G_ϕ tiene un k -clique.

Idea. ■ Si ϕ es satisfacible, alguna asignación hace verdadero al menos un literal por cláusula; los nodos correspondientes forman un k -clique.

■ Si G_ϕ tiene un k -clique, este debe tener exactamente uno por cláusula (no se pueden dos del mismo grupo) y ninguno contradictorio. Asignando 1 a los literales correspondientes se satisface ϕ .

□

Construcción polinomial. **CLIQUE es NP-completo.**

16.2. $3\text{-SAT} \leq_P \text{VERTEX-COVER}$

Definición 16.2 (Cubrimiento de vértices). Dado $G = (V, E)$ y un entero $k \leq |V|$, un **cubrimiento de vértices** de tamaño k es un $V' \subseteq V$ con $|V'| = k$ tal que cada arco de G tiene al menos un extremo en V' . El lenguaje es

$$\text{VERTEX-COVER} = \{ \langle G, k \rangle \mid G \text{ tiene un cubrimiento de tamaño } k \}.$$

Cubrimiento en NP. El certificado es V' ; el verificador chequea que $|V'| = k$ y que cada arista toca V' .

Reducción $3\text{-SAT} \leq_P \text{VC}$. Dada ϕ con m variables y ℓ cláusulas, construimos G_ϕ con dos gadgets:

- **Gadget de variable.** Por cada variable x , dos nodos x y $\bar{x}$ unidos por una arista.
- **Gadget de cláusula.** Por cada cláusula, un triángulo con 3 nodos (uno por literal).
- **Conexión.** Cada nodo de un triángulo se conecta con el nodo de variable del mismo label.

Buscamos un cubrimiento de tamaño $k = m + 2\ell$.

Teorema 16.3. ϕ es satisfacible si y solo si G_ϕ tiene un cubrimiento de tamaño $m + 2\ell$.

Idea. ($\Rightarrow$) Dada una asignación satisfaciente, para cada par $(x, \neg x)$ tomamos el nodo del literal verdadero, y en cada triángulo tomamos los dos correspondientes a literales falsos.

($\Leftarrow$) Un cubrimiento de tamaño $m + 2\ell$ debe tener exactamente 1 por par y 2 por triángulo. Asignamos True a las variables del cubrimiento: como el nodo no seleccionado del triángulo se conecta con un literal del cubrimiento de variable, ese literal es verdadero bajo la asignación, satisfaciendo la cláusula.

□

16.3. 3-SAT $\leq_P$ HAMPATH

Sabemos que HAMPATH $\in$ NP. Para mostrar NP-completitud se reduce polinomialmente 3-SAT a HAMPATH: dada una fórmula ϕ en 3-CNF se construye un grafo G_ϕ que tiene un camino Hamiltoniano de s a t si y solo si ϕ es satisfacible. La construcción usa gadgets de variable (que codifican la asignación) y gadgets de cláusula (que fuerzan al camino a satisfacer cada cláusula). El resultado: **HAMPATH es NP-completo**.

16.4. Otros NP-completos clásicos

- **LPATH**: $\{\langle G, a, b, k \rangle \mid G \text{ contiene un camino simple de longitud } \geq k \text{ de } a \text{ a } b\}$.
- **3-COLOR**: $\{\langle G \rangle \mid G \text{ es coloreable con 3 colores}\}$.
- **SUBSET-SUM**: se prueba NP-completo reduciendo 3-SAT.
- **ISO**: se sabe que está en NP, pero su status de NP-completitud es un problema clásico; en años recientes se han presentado algoritmos cuasi-polinomiales.

17. Propiedades de clausura

- P es cerrado bajo *unión, intersección, concatenación y complemento*.
- NP es cerrado bajo *unión, intersección y concatenación*. No se sabe si es cerrado bajo *complemento* (esto equivale a $NP \stackrel{?}{=} \text{coNP}$).

18. Complejidad espacial

Cuando analizamos algoritmos hay *dos dimensiones* importantes:

- **Tiempo**: cantidad de instrucciones básicas que se realizan.
- **Espacio**: cantidad de memoria que necesita el algoritmo.

Usamos MTS; la memoria se mide como la **cantidad de celdas de cinta** que la MT inspecciona.

18.1. Definición formal

Definición 18.1 (Complejidad espacial). Sea M una MT que siempre termina. La complejidad espacial de M es la función $f : \mathbb{N} \rightarrow \mathbb{N}$ donde

$f(n)$ = cantidad máxima de celdas que M mira para cualquier entrada de longitud n .

Si M es no determinista, $f(n)$ es la cantidad máxima de celdas inspeccionadas en **cualquier rama**.

18.2. Clases de complejidad espacial

Definición 18.2 (Clases de espacio).

$$\text{SPACE}(f(n)) = \{L \mid L \text{ es decidido por una MT en espacio } O(f(n))\}.$$

$$\text{NSPACE}(f(n)) = \{L \mid L \text{ es decidido por una MT no determinista en espacio } O(f(n))\}.$$

$$\text{PSPACE} = \bigcup_{k \geq 1} \text{SPACE}(O(n^k)), \quad \text{NSPACE} = \bigcup_{k \geq 1} \text{NSPACE}(O(n^k)).$$

18.3. SAT $\in$ PSPACE

Los algoritmos pueden *reusar* espacio pero no pueden reusar tiempo. Veamos:

Sea M la MT tal que dado ϕ :

1. Para cada asignación de valores de verdad de ϕ :
2. Evaluar ϕ en la asignación.
 - Si se evalúa a true, aceptar.
 - Sino limpiar la porción de memoria usada y continuar.
3. Rechazar.

Solo necesita una cantidad **lineal** de espacio (una asignación a la vez). $\Rightarrow$ **SAT** $\in$ PSPACE.

18.4. $\overline{\text{ALL}_{\text{NFA}}} \in \text{NPSPACE}$

$$\overline{\text{ALL}_{\text{NFA}}} = \{\langle A \rangle \mid A \text{ es un NFA y } L(A) \neq \Sigma^*\}.$$

N sobre input $\langle M \rangle$ con M un NFA:

1. Poner una marca en el estado inicial.
2. Repetir $2^{|Q|}$ veces:
 - No deterministamente elegir un símbolo y cambiar las marcas para simular M .
3. Si no se marcaron estados de aceptación, aceptar; sino rechazar.

Si hay una cadena que A no acepta, hay una cadena de longitud a lo sumo $2^{|Q|}$ no aceptada. Solo se necesita una cantidad **lineal** de espacio (las marcas).

19. El Teorema de Savitch

Teorema 19.1 (Savitch). $\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f(n)^2)$.

Es decir, toda MT no determinista que usa $f(n)$ espacio puede simularse por una MT determinista que usa a lo sumo $f(n)^2$ espacio.

Observación 19.2. No podemos simular el árbol completo porque cada rama podría tener longitud exponencial $f(n) \cdot 2^{O(f(n))}$ y eso costaría espacio exponencial.

Propiedad útil: una MT que usa $O(f(n))$ espacio puede realizar a lo sumo $f(n) \cdot 2^{O(f(n))}$ pasos antes de empezar a repetir configuraciones.

19.1. El algoritmo canyfield

La prueba consiste en definir un programa recursivo

$$\text{canyield}(c_1, c_2, t)$$

que dice: ¿se puede ir de la configuración c_1 a la configuración c_2 en a lo sumo t pasos, usando poco espacio?

canyield(c1, c2, t):

1. Si $t = 1$, verificar si $c1 = c2$ o si en un paso se llega de $c1$ a $c2$; aceptar o rechazar segun esto.
2. Si $t > 1$, para cada configuracion cm :
 3. Ejecutar canyild(c1, cm, $t/2$)
 4. Ejecutar canyild(cm, c2, $t/2$)
 5. Si los pasos 3 y 4 aceptan, aceptar.
6. Si no se acepto para ninguna, rechazar.

19.2. Análisis

Corremos canyild($c_0, q_{\text{accept}}, 2^{d \cdot f(n)}$) para alguna constante d :

- La MT original puede realizar a lo sumo $2^{O(f(n))}$ pasos: si acepta, el algoritmo aceptará.
- La pila de recursión solo puede tener tamaño **logarítmico** (t se divide por 2 en cada llamada): $\log 2^{d \cdot f(n)} = d \cdot f(n) \in O(f(n))$.
- En cada llamada utilizamos $O(f(n))$ celdas para la configuración.
- Espacio total: configuraciones $\times$ pila = $O(f(n)) \times O(f(n)) = O(f(n)^2)$.

19.3. Consecuencias

Corolario 19.3. $\text{NPSPACE} = \text{PSPACE}$.

Demostración. Si $L \in \text{NPSPACE}$, $L \in \text{NSPACE}(O(n^k))$ para algún k . Por Savitch, $L \in \text{SPACE}(O(n^{2k})) \subseteq \text{PSPACE}$. La inclusión recíproca es trivial. $\square$

Idea clave

Para el *espacio* polinomial, el no determinismo no ayuda. Esto contrasta con el caso del *tiempo*, donde no se sabe si $P = NP$.

20. Jerarquía $P \subseteq NP \subseteq \text{PSPACE} \subseteq \text{EXPTIME}$ y PSPACE -Completitud

20.1. Cadena de inclusiones

$$P \subseteq NP \subseteq \text{PSPACE} = \text{NPSPACE} \subseteq \text{EXPTIME}.$$

- $P \subseteq NP$: ya visto.
- $NP \subseteq \text{PSPACE}$: una MT no determinista en tiempo polinomial usa solo espacio polinomial; alternatively, podemos simular cada certificado posible reusando espacio (ejemplo SAT).
- $\text{PSPACE} = \text{NPSPACE}$: Teorema de Savitch.
- $\text{NPSPACE} \subseteq \text{EXPTIME}$: una MT con $O(f(n))$ espacio realiza a lo sumo $f(n) \cdot 2^{O(f(n))}$ pasos antes de repetir configuración. Si f es polinomial, esto da tiempo exponencial.

No se sabe cuáles de las inclusiones son estrictas: $P \stackrel{?}{=} NP$ y $NP \stackrel{?}{=} \text{PSPACE}$ son problemas abiertos.

20.2. PSPACE-Compleitud

Definición 20.1 (PSPACE-Compleitud). Un problema B se dice **PSPACE-completo** si:

1. $B \in \text{PSPACE}$.
2. Para todo $A \in \text{PSPACE}$, $A \leq_P B$.

Observación 20.2. También usamos *reducibilidad polinomial* (no espacial): si un procedimiento corre en tiempo polinomial, también corre en espacio polinomial.

21. Ideas clave para repasar

1. **Notaciones** O, o, Ω, Θ . Hablan de crecimiento asintótico módulo constantes. o es estricto, Θ es ajustada.
2. **Mismo problema, distinto tiempo.** El lenguaje $0^k 1^k$ tiene algoritmos $O(n^2)$, $O(n \log n)$, $O(n)$ con dos cintas. Modelo y astucia importan.
3. **La clase P.** $P = \bigcup_k \text{TIME}(O(n^k))$. Es robusta (Tesis de Church–Turing Extendida). Sus problemas se consideran *tratables*.
4. **Multicinta vs una cinta.** La simulación cuesta cuadrático ($O(t(n)^2)$), pero preserva polinomial.
5. **Tiempo en MT no determinista.** Es el largo de la rama más larga. La simulación determinista cuesta $2^{O(t(n))}$.
6. **Patrón “adivinar + verificar”.** Para mostrar $L \in \text{NP}$ basta exhibir un certificado polinomial y un algoritmo de verificación polinomial.
7. **Dos visiones equivalentes de NP.** Operacional (MT no determinista en tiempo polinomial) y declarativa (verificador polinomial con certificado polinomial).
8. $P \subseteq \text{NP}$. Trivial (verificador que ignora c). Pregunta abierta: $P \stackrel{?}{=} \text{NP}$.
9. **Reducibilidad $\leq_P$.** Es transitiva. Si $A \leq_P B$ y $B \in P$ (resp. NP), entonces $A \in P$ (resp. NP).
10. **NP-Compleitud.** L es NP-completo si $L \in \text{NP}$ y todo $A \in \text{NP}$ se reduce a L . Si uno de ellos está en P, entonces $P = \text{NP}$.
11. **Cook–Levin.** SAT es el primer NP-completo. La prueba codifica una rama aceptante de una MT no determinista como una asignación booleana mediante $\phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$. Tamaño $O(n^{2k})$.
12. **Cadena de reducciones.**
$$\text{SAT} \leq_P 3\text{-SAT} \leq_P \text{CLIQUE, VERTEX-COVER, HAMPATH, } \dots$$
13. **Método estándar.** Para ver que un nuevo L es NP-completo: (i) mostrar $L \in \text{NP}$; (ii) reducir polinomialmente algún NP-completo conocido a L .
14. **Espacio.** Se puede *reusar*; el tiempo no. $\text{SAT} \in \text{PSPACE}$ aunque sea exponencial en tiempo (en el peor caso conocido).
15. **Teorema de Savitch.** $\text{NSPACE}(f) \subseteq \text{SPACE}(f^2)$. En particular $\text{NPSPACE} = \text{PSPACE}$. Prueba vía *canyield* con pila logarítmica.

16. **Jerarquía.** $P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXPTIME$. Ninguna inclusión intermedia se sabe estricta.

22. Preguntas típicas de examen

Q1. Verdadero o falso.

	Afirmación	Justificación
a	$2n \in O(n)$ (V)	Con $c = 2$, $n_0 = 1$.
b	$n^2 \in O(n)$ (F)	$n^2/n = n \rightarrow \infty$.
c	$n^2 \in O(n \log^2 n)$ (F)	$n^2/(n \log^2 n) = n/\log^2 n \rightarrow \infty$.
d	$3^n \in 2^{O(n)}$ (V)	$3^n = 2^{n \log_2 3} \in 2^{O(n)}$.
e	$2^n \in o(3^n)$ (V)	$(2/3)^n \rightarrow 0$.
f	$1 \in o(1/n)$ (F)	$1/(1/n) = n \rightarrow \infty$.

Q2. ¿Por qué el algoritmo ingenuo para Coprimos no es polinomial? Porque los números vienen en binario: la entrada tiene tamaño $\log_2 x$. Recorrer hasta $\min(x, y)$ son 2^n pasos con n bits. El algoritmo eficiente (Euclides) divide por 2 en cada paso, dando $O(n \cdot p(n))$.

Q3. ¿Es P cerrada bajo unión, intersección, concatenación y complemento? Sí. Unión/intersección: ejecutar ambos algoritmos. Concatenación: probar los $|w| + 1$ cortes. Complemento: invertir aceptación.

Q4. ¿Por qué $P \subseteq NP$? Si $L \in P$, hay M determinista polinomial; el verificador V ignora el certificado y simula $M(w)$.

Q5. ¿Por qué CLIQUE está en NP? Certificado: subconjunto c . Verificador: chequear $|c| = k$, $c \subseteq V$, y que cada par en c esté conectado.

Q6. Si un NP-completo está en P, ¿qué se concluye? $P = NP$, porque todo $A \in NP$ se reduce al NP-completo y eso preserva P.

Q7. Idea de la reducción $SAT \leq_P 3\text{-SAT}$. Cada cláusula de tamaño n se reescribe usando $n - 2$ variables nuevas z_j :

$$(x_0 \vee x_1 \vee z_1) \wedge (\neg z_1 \vee x_2 \vee z_2) \wedge \cdots \wedge (\neg z_{n-2} \vee x_{n-2} \vee x_{n-1}).$$

Equisatisfacible y polinomial.

Q8. ¿Qué es una “ventana legal” en Cook–Levin? Una ventana de 2×3 celdas (3 en fila i , 3 en fila $i + 1$) cuyo contenido es consistente con alguna transición de N . Si todas las ventanas son legales, la tabla describe una rama válida.

Q9. Enunciar Savitch y su consecuencia. $NSPACE(f(n)) \subseteq SPACE(f(n)^2)$. Consecuencia: $PSPACE = NPSPACE$. Prueba: `canyield` recursivo con pila logarítmica.

Q10. ¿Por qué $SAT \in PSPACE$? Se recorren todas las asignaciones reusando el mismo bloque de memoria. Espacio lineal.

Q11. Tesis de Church–Turing Extendida. Cualquier modelo razonable determinista que decide un problema en $O(t(n))$ es simulable en $O(t(n)^k)$ por cualquier otro modelo razonable determinista.

Q12. ¿Por qué “Path” está en P? Algoritmo de BFS: marcar v y propagar a los vecinos hasta cubrir todos los nodos alcanzables. $O(|V| \cdot |E|)$.

23. Resumen ultra breve

- **Complejidad** estudia el tiempo y el espacio que requieren los algoritmos. Notaciones O, o, Ω, Θ describen crecimiento asintótico módulo constantes.
- **Clase P:** problemas decidibles en tiempo polinomial por una MT determinista. Tratables. Invariante respecto a modelos deterministas razonables (Tesis Church–Turing Extendida).
- **Clase NP:** problemas decidibles en tiempo polinomial por una MT *no determinista* $\equiv$ problemas con *verificador polinomial* (certificado corto, verificación rápida). Patrón “adivinar + verificar”. Ejemplos: HAMPATH, CLIQUE, SUBSET-SUM, COMP.
- **Reducibilidad** $A \leq_P B$: existe f polinomial con $w \in A \iff f(w) \in B$. Transitiva. Preserva P y NP.
- **NP-Completo:** los más difíciles de NP. **Cook–Levin:** SAT es NP-completo. Reducciones clásicas: $\text{SAT} \leq_P \text{3-SAT} \leq_P \text{CLIQUE}$, VERTEX-COVER, HAMPATH.
- **Espacio:** se puede *reusar*; el tiempo no. $\text{SAT} \in \text{PSPACE}$. **Savitch:** $\text{NSPACE}(f) \subseteq \text{SPACE}(f^2)$, por lo cual $\text{PSPACE} = \text{NPSPACE}$.
- **Jerarquía:** $\text{P} \subseteq \text{NP} \subseteq \text{PSPACE} = \text{NPSPACE} \subseteq \text{EXPTIME}$. Las inclusiones intermedias se conjeturan estrictas pero ninguna se ha demostrado.